KukuFM PromoFinder

Problem 1: Find the Promos

Problem choice and approach

I built a prototype that turns timestamped screenplay episodes into ranked, reviewable promo candidates. It is designed around three tests: a new viewer must understand the moment, the opening must create urgency in the first three seconds, and the end must leave a question unresolved.

How the system works

01 Parse
Read episode files and timestamped beats.

02 Generate
Create candidate windows of 20 to 60 seconds.

03 Score
Score hooks, cliffhangers, clarity, and spoiler risk.

04 Validate
Check duration, quote, beat boundaries, and overlap.

05 Review
Let the editor accept, reject, or nudge every result.

Evidence from real runs

50	300	30	15.4s
episodes parsed	candidates generated	promos ranked	heuristic run time

What is automated and what stays human

Code owns: file parsing, candidate timing, duration checks, beat-edge snapping, exact quote verification, overlap rejection, exports, and repeatable runs. **AI assists:** compact candidate judgments when configured. **The editor owns:** cultural fit, social appeal, final cuts, and accept or reject decisions.

Quality, scale, and honest limits

Measured evidence and next release checks

Quality checks and observed results

30 / 30	promos passed duration, quote, boundary, and validation checks
52 / 52	automated tests passed
98 / 100	full-pack preflight health; one sound block exceeded its scene
2 / 9	model smoke judgments matched exact source quote evidence

Known failures

A rule-valid candidate is not guaranteed to be a good promo. A first line can be grammatically strong while the visual action lacks urgency. The first full live run also stopped at global ranking with a code error. It was not used as a completed AI ranking. The smallest repeatable sample and the full heuristic run are published separately, and cold-viewer labels are not claimed as completed.

How it scales

A show is processed episode by episode with bounded LLM calls and resumable cache. A new show needs only its screenplay files. Provider and model names are configuration. At production volume, I would add a worker queue, object storage, per-show isolation, editor feedback labels, and a publishing review gate. The current heuristic run cost was 0 because it used no paid provider calls.

AI tooling and one observed mistake

Tools used: Python, Streamlit, FastAPI, Groq, Vercel, GitHub, Pytest, and ReportLab. One observed model error was valid JSON and plausible timestamps paired with changed screenplay wording. The exact quote verifier rejected it. This is why the final evidence checks remain outside the model.

Submission links

Prototype

https://kukufm-promofinder-live.vercel.app

Repository

https://github.com/elprofessor-15/KukuFM